

Software Design Document

AttentionPay

Interpretable Transformer Model for Explainable Fraud Detection

Clean final draft with updated architecture, scenario-based demo data,
formatted tables, and visual flowcharts

1. Project Overview

1.1 Project Title

- AttentionPay: Interpretable Transformer Model for Explainable Fraud Detection

1.2 Problem Statement

- Online payment fraud and phishing attacks are increasing due to rapid growth of digital payment systems.
- Existing fraud detection systems often behave like black boxes, making it difficult to understand why a transaction is marked fraudulent or safe.
- AttentionPay builds a secure, explainable, multi-layer fraud detection simulator that detects phishing before payment, validates backend security rules, detects behavioural fraud using AI, and explains every final result.

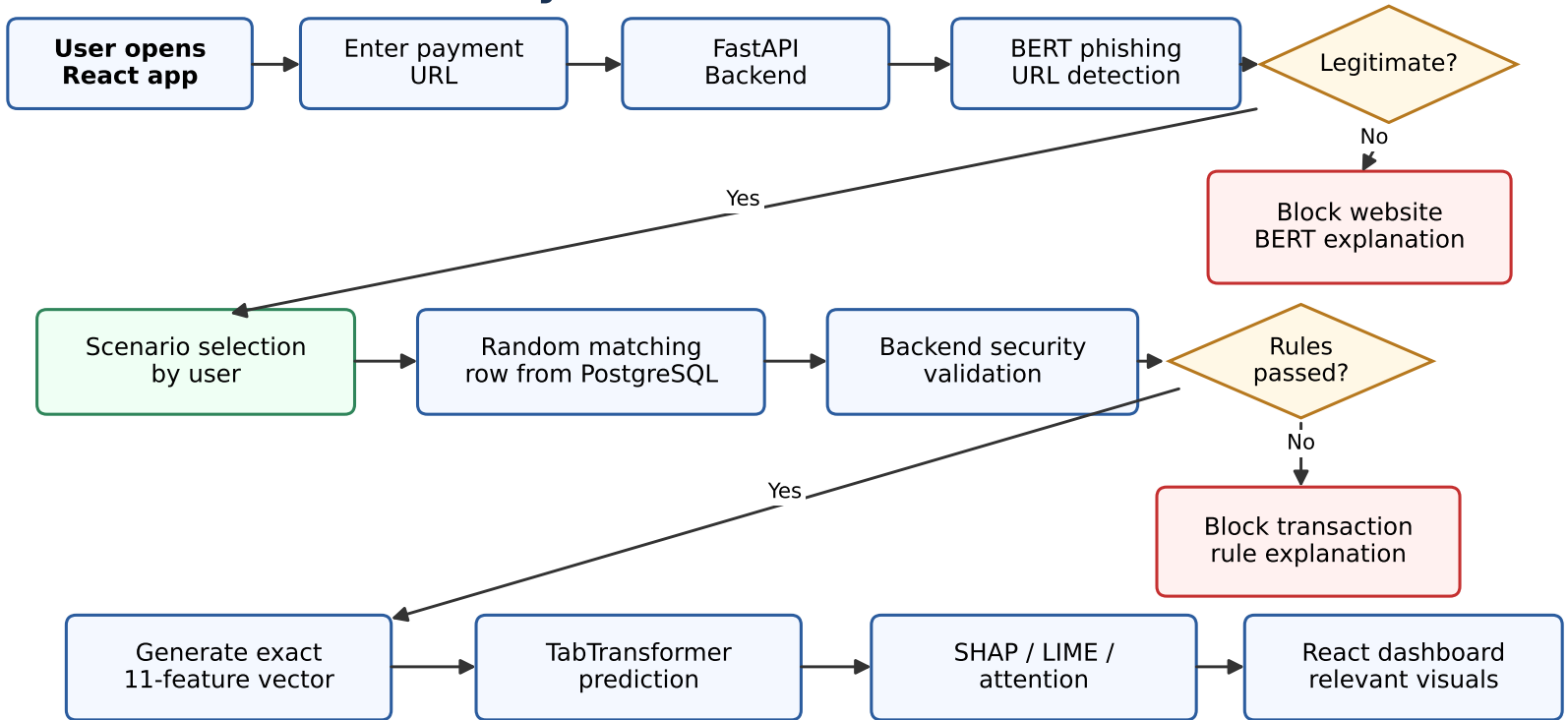
1.3 Objectives

- Detect phishing payment URLs using a fully trained BERT Transformer model.
- Prevent users from continuing to phishing payment pages.
- Simulate online payments without using real banking infrastructure.
- Validate transactions using backend security checks.
- Detect behavioural transaction fraud using a fully trained TabTransformer model.
- Explain every final result: blocked, suspicious, or successful.
- Build the frontend using React.js, JavaScript, and Axios.
- Use PostgreSQL to store default demo user, scenario rows, transaction runs, model outputs, and explanations.

1.4 Scope

- Includes phishing URL detection, React frontend, scenario-based simulator, backend validation, TabTransformer fraud detection, Explainable AI, PostgreSQL, and dashboard visualizations.
- Does not include real bank accounts, real payment gateways, real money transfer, live banking APIs, or production banking deployment.

Flowchart 1: Overall System Architecture



3. Technology Stack

Frontend

- React.js
- JavaScript
- Axios for API communication
- Recharts or Chart.js for charts
- Leaflet.js for impossible travel map visualization

Backend

- FastAPI
- Uvicorn
- Pydantic
- SQLAlchemy
- Alembic

Database

- PostgreSQL

AI/ML

- BERT using HuggingFace Transformers
- TabTransformer using PyTorch
- SHAP
- LIME
- Attention visualization
- Pandas, NumPy, Scikit-learn

4. Layer 1: Phishing URL Detection

Purpose

- Checks whether a payment URL is legitimate or phishing before allowing scenario simulation.

Model

- Fully trained BERT Transformer model.

Dataset

- Recommended dataset: PhiUSIIL Phishing URL Dataset from UCI.
- Use URL text and label only for BERT training.
- Balance training data by sampling equal legitimate and phishing URLs.
- Recommended split: 80% training, 10% validation, 10% testing.

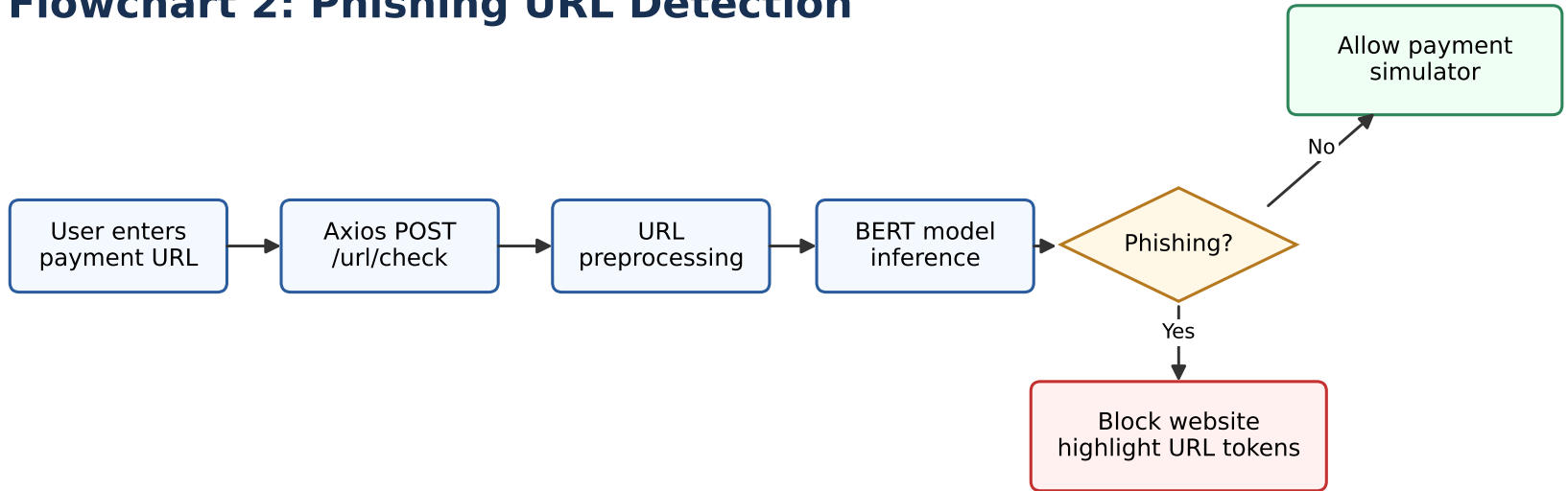
Input

- Payment URL entered by user. Example: <https://amazon.in>. Example phishing URL:
- <http://paypal-login-security.xyz>.

Output

- Prediction: legitimate or phishing.
- Confidence score.
- Explanation using highlighted suspicious URL tokens and optional attention heatmap.

Flowchart 2: Phishing URL Detection



5. Online Payment Simulator

5.1 Purpose

- Demonstrates fraud detection without real banks, real payment gateways, real OTPs, or real money movement.

5.2 Payment Simulator Workflow

- URL is classified as legitimate.
- React opens the payment simulator.
- User selects only the transaction scenario.
- Backend randomly selects one matching demo transaction row from PostgreSQL.
- Frontend displays selected transaction details as read-only preview.
- User clicks Simulate or Analyze Transaction.
- Backend performs security validation.
- If backend passes, the 11-feature vector is generated and sent to TabTransformer.
- Final result and explanation are returned to the dashboard.

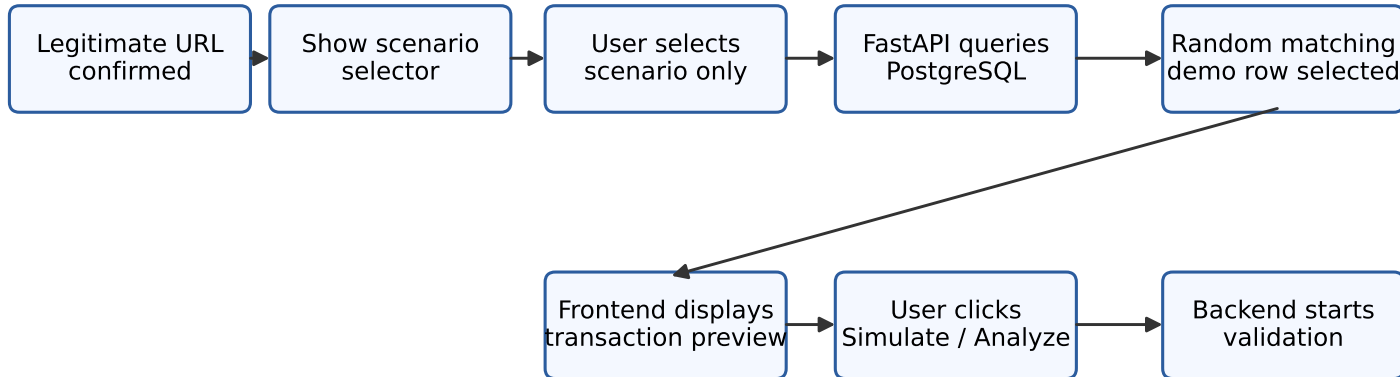
5.3 User Inputs

- Selected scenario.
- Simulate/Analyze Transaction button click.

5.4 Backend-Fetched Demo Data

- Receiver identifier.
- Transaction amount.
- Previous transaction amount.
- Previous and current location.
- Previous and current transaction time.
- Device, browser, and operating system.
- Expected and actual API endpoint.
- SSL status and authentication status.
- Velocity values and session risk score.

Flowchart 3: Scenario-Based Payment Simulation



User does not enter receiver, amount, PIN, or OTP.
These values come from the selected demo transaction row.

6. Transaction Scenarios

6.1 Normal Transaction

- Known device.
- Normal transaction amount.
- Low transaction velocity.
- Low session risk.
- Possible travel.
- Correct API endpoint.
- Valid SSL.
- Authentication passed.
- Expected: backend passes, TabTransformer predicts safe, transaction successful, explanation shown.

6.2 Impossible Travel

- Previous and current locations are far apart.
- Time difference is too small.
- Required travel speed is unrealistic.
- Expected: backend blocks transaction, TabTransformer is not executed, backend explanation and map visualization are shown.

6.3 API Route Tampering

- Expected endpoint is `/api/payment`.
- Actual endpoint may be `/api/fake-payment`, `/api/testing`, or `/attacker/payment`.
- Expected: backend blocks transaction and route mismatch is explained.

6.4 Behaviour Fraud

- Unknown device.
- High transaction velocity.
- Large transaction amount.
- High session risk.
- Backend checks pass.
- Expected: TabTransformer generates fraud probability and AI explanation.

7. Scenario Generator and Scenario Database

Purpose

- The database stores multiple demo rows for each scenario.
- When the user selects a scenario, FastAPI randomly selects one matching row.

Demo Scenario Distribution

- 25% normal_transaction.
- 25% impossible_travel.
- 25% api_route_tampering.
- 25% behaviour_fraud.

TabTransformer Training Distribution

- 50% normal_transaction.
- 50% behaviour_fraud.
- Impossible travel and API route tampering are backend-blocked and should not train the TabTransformer.

Default User Design

- No user-facing registration or login.
- Database keeps one internal default demo user for relationships.
- Demo transaction rows are linked to this default user.

8. Layer 2: Backend Security Validation

8.1 Purpose

- Performs deterministic security checks before AI fraud detection.
- If a backend rule fails, transaction is blocked immediately and explained.

8.2 Backend Checks

- Impossible Travel Check.
- API Routing Verification.
- SSL Verification.
- Authentication Verification.

8.3 Impossible Travel Check

- Inputs: previous latitude, previous longitude, current latitude, current longitude, previous transaction time, current transaction time.
- Calculates distance, time difference, and required travel speed.
- If required speed exceeds 900 km/h, transaction is blocked.

8.4 API Routing Verification

- Inputs: expected API endpoint and actual API endpoint.
- If actual endpoint does not match expected endpoint, transaction is blocked.

8.5 SSL Verification

- Input: ssl_valid.
- If false, transaction is blocked.

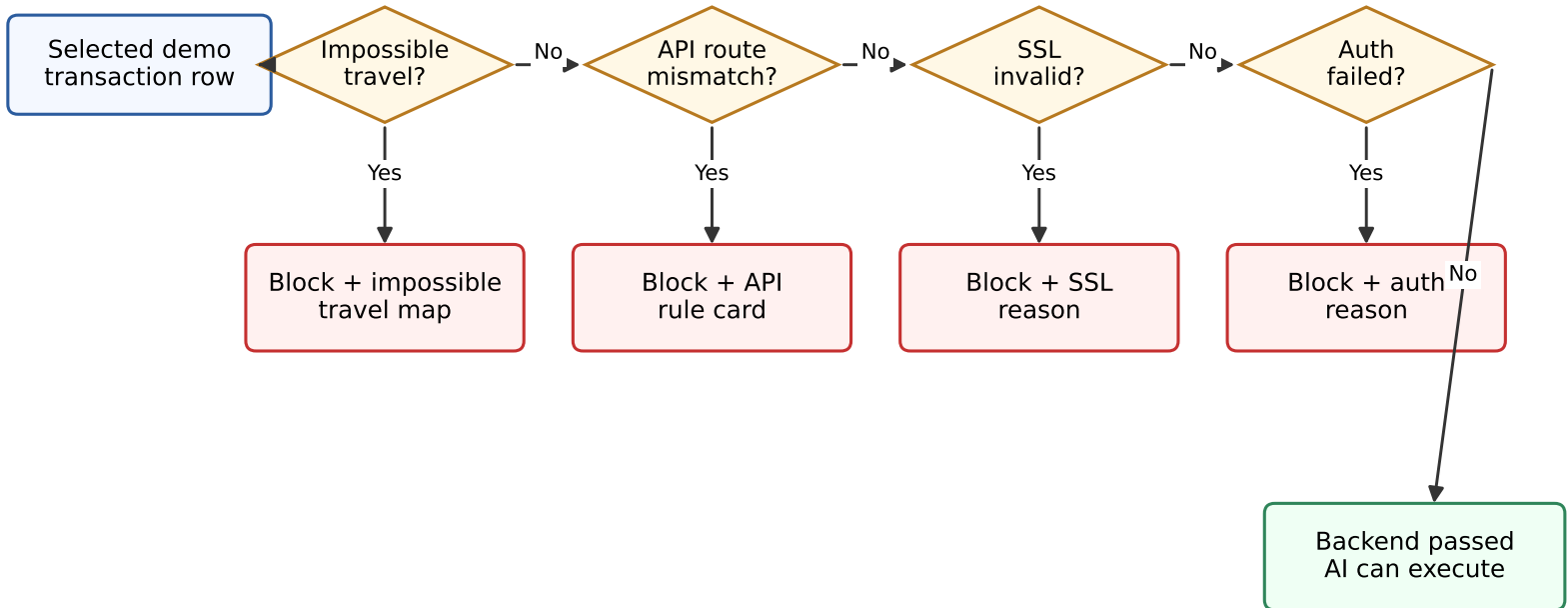
8.6 Authentication Verification

- Input: authentication_passed.
- If false, transaction is blocked.

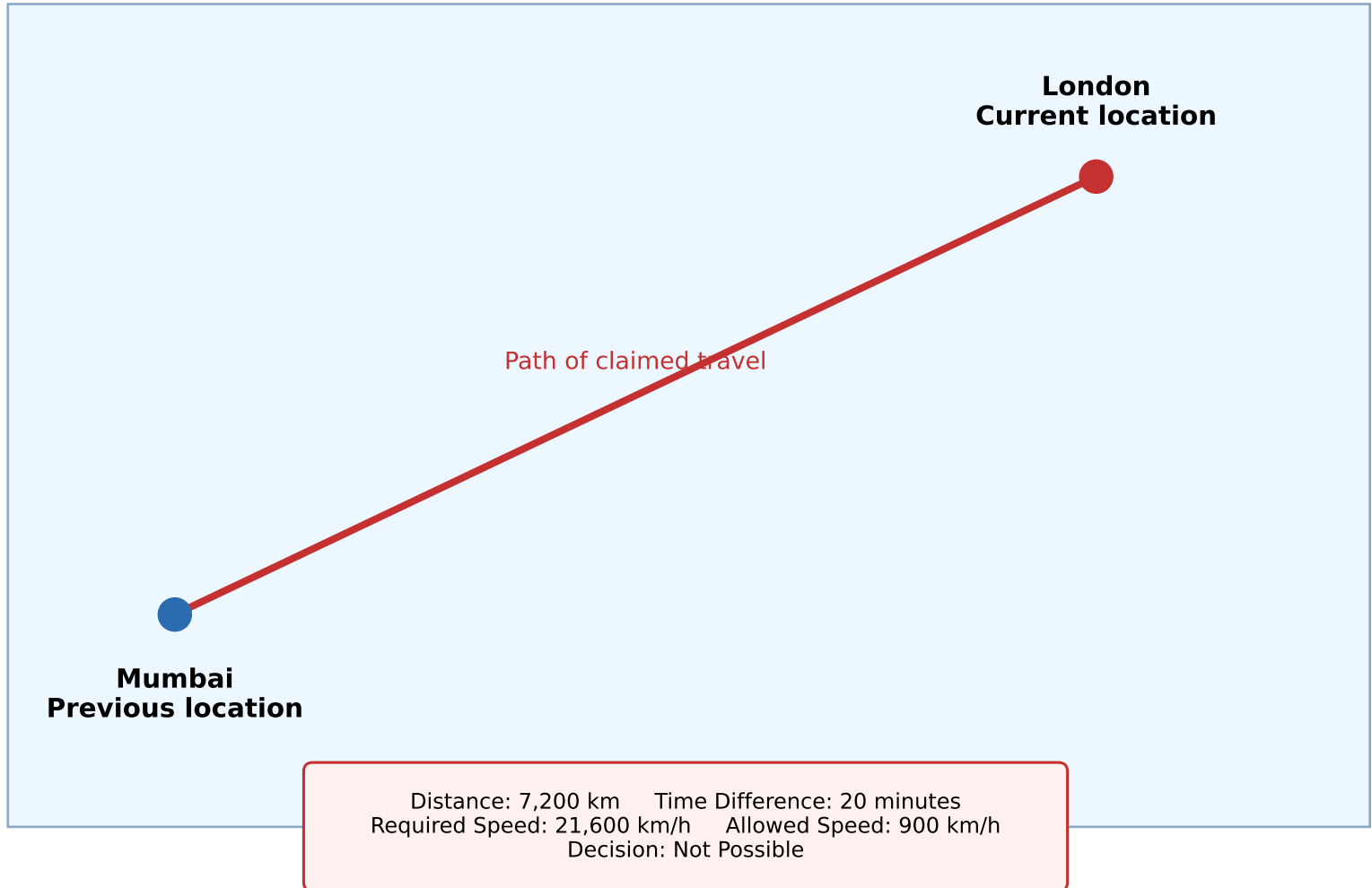
8.7 Backend Output Format

Blocked by backend	backend_status: blocked backend_block_reason: impossible_travel / api_route_tampering / ssl_invalid / authentication_failed ai_executed: false final_status: blocked explanation_source: backend_rule_engine
Passed backend	backend_status: passed backend_block_reason: null ai_executed: true final_status: pending_ai_result next_step: feature_engineering

Flowchart 4: Backend Security Validation



Flowchart 7: Impossible Travel Map Visualization



9. Feature Engineering

Purpose

- Converts a backend-passed demo transaction row into the exact 11-feature vector required by
- TabTransformer.

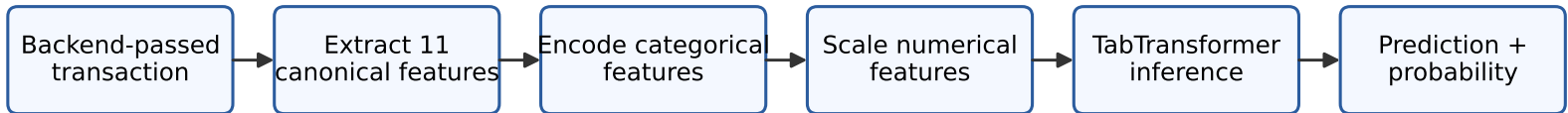
Compatibility Rule

- These feature names must remain fixed across database, backend API, training CSV, model
- inference, explainability output, and React dashboard.

9.1 Canonical 11-Feature Vector

Feature Name	Type	Meaning
transactions_last_1min	Numerical	Number of transactions in the last 1 minute.
transactions_last_5min	Numerical	Number of transactions in the last 5 minutes.
transactions_last_10min	Numerical	Number of transactions in the last 10 minutes.
transaction_amount	Numerical	Current transaction amount.
previous_transaction_amount	Numerical	Previous transaction amount.
known_device_flag	Categorical	1 if device is known, otherwise 0.
device_changed_flag	Categorical	1 if device changed, otherwise 0.
device_type	Categorical	mobile, laptop, or tablet.
browser_name	Categorical	chrome, firefox, edge, or safari.
operating_system	Categorical	android, ios, windows, macos, or linux.
session_risk_score	Numerical	Risk score from current session behaviour.

Flowchart 5: Feature Engineering and TabTransformer Pipeline



Numerical
transactions_last_1min
transactions_last_5min
transactions_last_10min
transaction_amount
previous_transaction_amount
session_risk_score

Categorical
known_device_flag
device_changed_flag
device_type
browser_name
operating_system

Explainability
SHAP
LIME
Attention visualization

10. Behavioural Fraud Detection

Purpose

- Detects behavioural fraud that cannot be caught by simple backend rules.

Model

- Fully trained TabTransformer.

Dataset Strategy

- Use synthetic AttentionPay transaction data because the required 11-feature vector is
- custom.
- Public fraud datasets can inspire ranges but usually do not provide all required fields.

Output

- Prediction: safe or suspicious.
- Fraud probability.
- Explanation using SHAP, LIME, and attention visualization.

1.1. Exact TabTransformer Training Data Format

Column	Used by Model?	Description
transaction_id	No	Metadata identifier.
user_id	No	Default demo user identifier.
scenario_name	No	normal_transaction or behaviour_fraud for training.
transactions_last_1min	Yes	Numerical feature.
transactions_last_5min	Yes	Numerical feature.
transactions_last_10min	Yes	Numerical feature.
transaction_amount	Yes	Numerical feature.
previous_transaction_amount	Yes	Numerical feature.
known_device_flag	Yes	Categorical feature.
device_changed_flag	Yes	Categorical feature.
device_type	Yes	Categorical feature.
browser_name	Yes	Categorical feature.
operating_system	Yes	Categorical feature.
session_risk_score	Yes	Numerical feature.
label	Target	0 = safe, 1 = suspicious/fraud.

11.1 Example Training Rows

transaction_id	scenario_name	1m	5m	10m	amount	prev_amount	known	changed	device	browser	os	risk	label
1	normal_transaction	0	1	2	750.00	680.00	1	0	mobile	chrome	android	0.10	0
2	normal_transaction	0	1	1	1250.00	1100.00	1	0	laptop	edge	windows	0.15	0
3	behaviour_fraud	3	8	15	65000.00	1200.00	0	1	mobile	chrome	android	0.88	1
4	behaviour_fraud	2	7	12	42000.00	900.00	0	1	laptop	firefox	windows	0.76	1

Safe transaction

transactions_last_1min: 0-1
transactions_last_5min: 0-3
transactions_last_10min: 1-5
transaction_amount: INR 100-5,000
previous_transaction_amount: similar to current amount
known_device_flag: 1
device_changed_flag: 0
session_risk_score: 0.00-0.35
label: 0

Behaviour fraud

transactions_last_1min: 2-6
transactions_last_5min: 5-15
transactions_last_10min: 10-30
transaction_amount: INR 20,000-100,000
previous_transaction_amount: usually much lower
known_device_flag: 0
device_changed_flag: 1
session_risk_score: 0.60-1.00
label: 1

12. Synthetic Fraud Data Generation Rules

13. Explainable AI

Purpose

- Explains why the system produced the final result.
- Explanation is mandatory for every final outcome.

Explanation Sources

- Phishing URL: BERT/XAI with highlighted URL tokens.
- Backend block: rule-based backend explanation.
- AI safe/fraud: SHAP, LIME, and attention visualization.

Examples

- Safe: legitimate URL, backend checks passed, known device, normal amount, low velocity, low session risk.
- Fraud: unknown device, high velocity, large amount, high session risk.
- Backend block: impossible travel or unexpected API endpoint with concrete values.

Flowchart/Table 6: Result Dashboard Display Logic

Phishing URL	Status Card URL Token Highlighting Explanation Panel Optional BERT Attention Heatmap
Impossible Travel	Status Card Backend Rule Card Impossible Travel Map Visualization Explanation Panel
API Route Tampering	Status Card Backend Rule Card
Normal Successful Transaction	Status Card Explanation Panel Fraud Probability Gauge Feature Contribution Bar Chart Explanation Panel Optional Attention Heatmap
Behaviour Fraud	Status Card Fraud Probability Gauge Feature Contribution Bar Chart Explanation Panel Optional Attention Heatmap

14. Frontend Design: React.js

Stack

- React.js.
- JavaScript.
- Axios.
- Recharts or Chart.js.
- Leaflet.js.

Pages

- Home / URL Check Page.
- Scenario Selection Page.
- Payment Simulator Page.
- Result Dashboard.

Components

- Navbar.
- URLInputForm.
- PhishingResultCard.
- ScenarioSelector.
- DemoTransactionPreview.
- SimulateButton.
- TransactionResultCard.
- ExplanationPanel.
- FraudProbabilityGauge.
- FeatureContributionChart.
- BackendRuleCard.
- URLTokenHighlighter.
- AttentionHeatmap.
- ImpossibleTravelMap.

15. Backend Design: FastAPI

Modules

- url_detection.
- scenario_service.
- backend_validation.
- feature_engineering.
- fraud_detection.
- explainability.
- database.
- api_routes.

API Endpoints

- POST /url/check.
- GET /scenarios.
- POST /transactions/simulate.
- GET /transactions/{transaction_id}.
- GET /transactions/{transaction_id}/explanation.
- GET /health.

Removed Auth Endpoints

- No user-facing /auth/register, /auth/login, or /auth/me endpoints are required.

15.1 Transaction API Response Format

Field	Example / Meaning
transaction_id	123
final_status	successful / blocked / suspicious
backend_status	passed / blocked
backend_block_reason	null / impossible_travel / api_route_tampering / ssl_invalid / authentication_failed
ai_executed	true / false
ai_prediction	safe / suspicious / null
fraud_probability	0.08 / 0.89 / null
explanation_source	bert_xai / backend_rule_engine / tabtransformer_xai
explanation.summary	Short human-readable result explanation.
explanation.reasons	List of reason strings.
visualization_payload	Data required by dashboard visual components.

16. Database Schema

users	id
payment_urls	username
	user_type
	created_at
	bert_prediction
	bert_confidence
	is_phishing
	explanation_json
	checked_at

scenario_templates	id
	scenario_name
	description
	should_fail_backend
	should_reach_ai
	expected_result

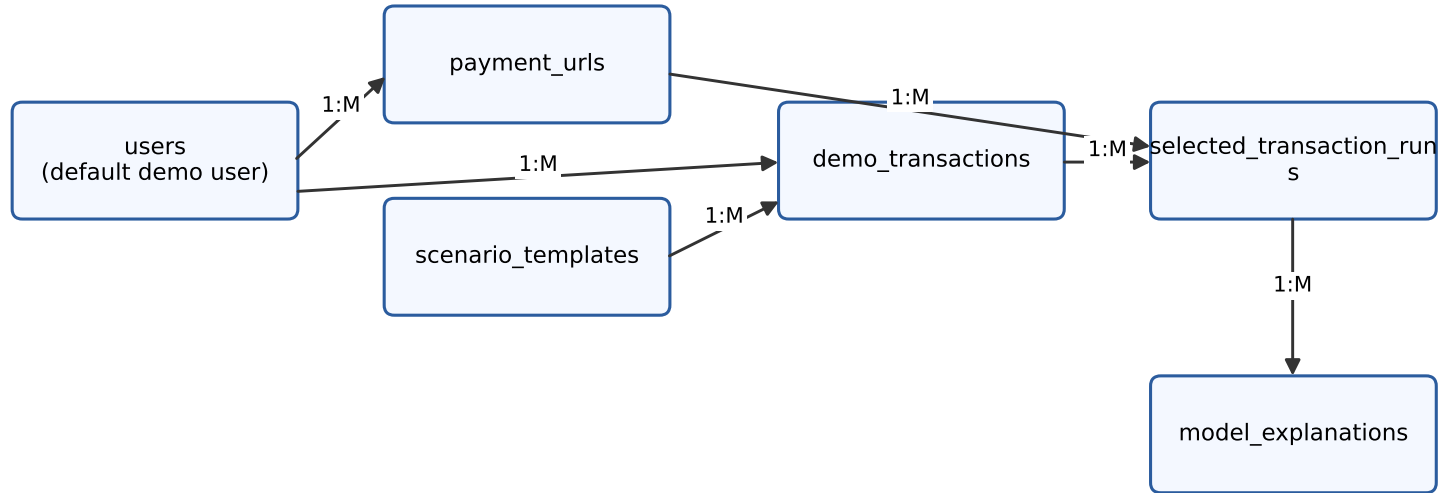
demo_transactions	id user_id scenario_id receiver_identifier transaction_amount previous_transaction_amount currency previous_city previous_country previous_latitude previous_longitude current_city current_country current_latitude current_longitude previous_transaction_time current_transaction_time expected_api_endpoint actual_api_endpoint ssl_valid authentication_passed transactions_last_1min transactions_last_5min transactions_last_10min known_device_flag device_changed_flag device_type browser_name operating_system session_risk_score label
-------------------	--

16. Database Schema (continued)

selected_transaction_runs	id demo_transaction_id payment_url_id backend_status backend_block_reason ai_executed fraud_probability ai_prediction final_status created_at
---------------------------	--

model_explanations	id transaction_run_id explanation_type model_name explanation_json created_at
--------------------	--

Flowchart 8: Database Entity Relationship Diagram



17. Complete System Workflow

End-to-End Workflow

- User opens React app.
- User enters payment URL.
- BERT performs phishing detection.
- If phishing, website is blocked and URL explanation is shown.
- If legitimate, scenario selector is shown.
- User selects scenario.
- Backend randomly selects matching demo transaction row.
- Frontend displays read-only transaction preview.
- User clicks Simulate or Analyze.
- Backend performs deterministic security validation.
- If backend rule fails, transaction is blocked and backend explanation is shown.
- If backend passes, exact 11-feature vector is generated.
- TabTransformer predicts safe or suspicious.
- SHAP/LIME/attention explanations are generated.
- React dashboard shows only relevant visuals.

18. Project Folder Structure

Recommended Structure

- backend/app/main.py, config.py, database.py.
- backend/app/models, schemas, routes, services, utils.
- frontend/src/pages, components, services, hooks, styles.
- frontend/src/App.jsx and main.jsx.
- datasets/phishing and datasets/fraud_synthetic.
- ml/bert_phishing.
- ml/tabtransformer.
- ml/explainability.
- docs, tests, docker-compose.yml, README.md.

19. Implementation Order

Phases

- Project setup.
- Database schema and seed default demo user, scenarios, and demo rows.
- Phishing dataset preparation and BERT training.
- Synthetic fraud dataset generation.
- TabTransformer training.
- Backend validation services.
- Scenario-based transaction simulator.
- Explainability integration.
- React frontend.
- Integration testing.
- Documentation.

20. Testing Strategy

Functional Testing

- URL checking.
- Scenario selection.
- Random transaction row selection.
- Backend validation.
- AI prediction.
- Explanation dashboard.
- Relevant visual rendering.

Scenario Testing

- Normal transaction: successful with safe explanation.
- Impossible travel: blocked with map and speed explanation.
- API route tampering: blocked with endpoint mismatch.
- Behaviour fraud: suspicious with probability and feature explanation.
- Phishing URL: blocked with URL token explanation.

AI Model Testing

- BERT: accuracy, precision, recall, F1-score, confusion matrix.
- TabTransformer: accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix.

21. Expected Outputs and Future Enhancements

Expected Outputs

- Legitimate website proceeds to simulator.
- Phishing website is blocked and explained.
- Backend-blocked transaction shows backend reason.
- Successful transaction shows safe explanation.
- Suspicious transaction shows fraud probability and AI explanation.

Future Enhancements

- Real banking APIs.
- Live payment gateway support.
- Real-time location tracking with permission.
- Multi-factor authentication.
- Graph Neural Networks.
- Continuous monitoring.
- Mobile application.
- Cloud deployment.
- Admin dashboard.
- Analyst review system.

22. Conclusion

Conclusion

- AttentionPay is a multi-layer explainable fraud detection simulator.
- It uses BERT for phishing URL detection, FastAPI for backend validation and ML inference,
- PostgreSQL for scenario-based demo data, and TabTransformer for behavioural fraud detection.
- The React.js frontend uses JavaScript and Axios, and it displays only the visuals relevant to the result.
- Every final outcome is explained, whether blocked by backend rules, marked suspicious by AI,
- or completed successfully.